FINAL COURSE PROJECT · APPLIED DATA ANALYTICS & MACHINE LEARNING BASICS

Loan Approval Prediction

Building and comparing three machine learning models to predict home-loan approval, from a raw dataset through to a working, leakage-safe pipeline.

Student	Abdulkadir Abdullahi Mohamed
Institution	Dhambaal Institute — Batch BDA01
Dataset	Loan Prediction Dataset — Kaggle (public, educational use)
Models	Logistic Regression · Decision Tree · K-Nearest Neighbours
Date	26 September 2026

1. Business Problem & Research Question 1
2. Dataset & Data Description 2
3. EDA Findings 2
4. Data Cleaning & Preprocessing 3
5. Feature Engineering 3
6. Machine Learning Models 3
7. Model Evaluation & Comparison 4
8. Key Findings 4
9. Limitations 5
10. Conclusion 5
11. Reflection 5

01

Business Problem & Research Question

When someone applies for a home loan, a bank officer reads the application and decides yes or no. The decision is slow, it is not always consistent between officers, and an applicant can wait days for an answer.

> **Research question:** Can we predict whether a home-loan application will be approved, using the applicant's income, credit history, loan amount and personal details?

Item	Decision
Problem type	Classification — the target is a category (approved / not approved), not a number
Target variable	`loan_status` (Y = approved, N = not approved)
Primary metric	F1-score, supported by accuracy, precision, recall and a confusion matrix

Why F1-score and not accuracy alone?

The target turned out to be imbalanced: about 69% of applications in this dataset were approved. A model that predicts “approved” for every single applicant would already score 69% accuracy while learning nothing. F1-score — the harmonic mean of precision and recall — only rewards a model that is both careful and thorough, so it was chosen as the metric that actually matters for this problem.

02

Dataset & Data Description

The dataset is the **Loan Prediction Problem Dataset**, a public dataset of home-loan applications originally published by Analytics Vidhya and widely mirrored on Kaggle.

Source: Loan Prediction Problem Dataset — Analytics Vidhya / Kaggle (`https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset`). Publicly available for educational use. Included with this submission as `dataset.csv`.

614	69% / 31%	22%	0
applications, 13 columns	approved vs. rejected — imbalanced	of rows had a missing value	duplicate rows found

The data mixes five numeric features (incomes, loan amount, loan term) with six categorical features (gender, marital status, education, self-employment, credit history, property area) — exactly the mix a real classification problem needs, plus genuine missing values that make the cleaning step meaningful rather than a formality.

03

EDA Findings

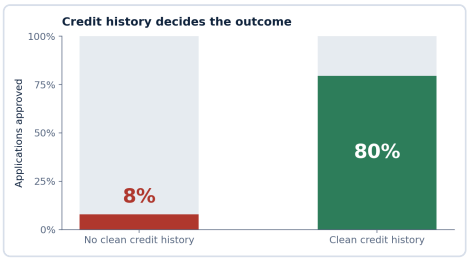


Figure 1. Applicants with a clean credit history are approved 80% of the time; those without, only 8%.

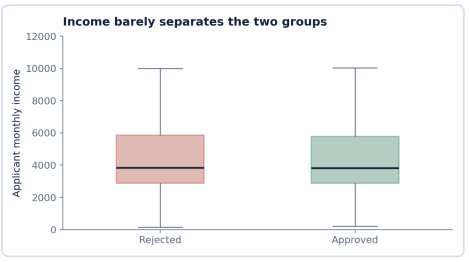


Figure 2. Applicant income for approved vs. rejected applications is almost identical.

What the exploration showed

- **Credit history dominates everything else** — a tenfold difference in approval rate between applicants with and without a clean record. No other feature comes close.
- **Income barely matters.** The income distributions for approved and rejected applicants overlap almost completely — a genuinely counter-intuitive finding.
- **Property area** has a modest effect (semiurban applicants approved more often than rural).
- The target is **imbalanced** (69% / 31%), and **22% of rows** have at least one missing value, concentrated most heavily in `credit_history` (50 missing — 8.1%), the single strongest predictor.

04

Data Cleaning & Preprocessing

Cleaning was split into two deliberate parts, because they carry different risks:

Part	What happens	Why it matters
Structural	Drop the ID column, fix `dependents` ("3+" → 3), encode the target	Uses no statistic calculated from the data — nothing to leak
Statistical	Fill missing values, one-hot encode, scale	These *learn* a value from the data and must be fitted on the training set only

THE MISTAKE I MADE, AND HOW IT WAS FIXED

My first version filled missing values on the **whole dataset before splitting** it into train and test. A mentor review flagged this: the median used to fill a gap in the test set had, in effect, been calculated with help from the test set itself — a subtle form of data leakage. The fix was to build a `ColumnTransformer` + `Pipeline` that is **fitted on the training set only**, and to run that same pipeline fresh inside every cross-validation fold. Section 11 (Reflection) describes this in more depth.

The final pipeline: numeric columns are filled with the training-set **median** (robust to the income outliers found in EDA) and scaled; categorical columns are filled with the training-set **most frequent value** and one-hot encoded. Everything is fitted on the 491 training applications only, then applied to the 123 held-out test applications.

05

Feature Engineering

Since EDA showed raw income barely separates the two classes, one new feature was built to test whether *affordability* (income relative to the loan) matters more than income alone: `total_income` = applicant income + co-applicant income. **Honest result:** this feature did not meaningfully change model performance — a finding reported rather than hidden, consistent with the EDA finding that credit history, not income, drives approval.

06

Machine Learning Models

Three models were trained on identical data and compared fairly:

Model	Why chosen
Logistic Regression	A readable baseline — its coefficients show exactly which factors drove a decision, important where lending decisions must be explainable
Decision Tree	Captures non-linear rules; the tree itself can be drawn and read by a non-technical reviewer
K-Nearest Neighbours (KNN)	A different philosophy entirely — makes no assumption about the shape of the relationship, useful as a contrast

Every tunable setting — Logistic Regression's `C`, the Decision Tree's `max_depth`, and KNN's `k` — was chosen the same way: with **5-fold cross-validation on the training set only**, never by looking at the test set. This keeps the three models comparable on equal, honest terms.

07

Model Evaluation & Comparison

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression — recommended	**86.18%**	**84.00%**	**98.82%**	**90.81%**
Decision Tree	84.55%	83.00%	97.65%	89.73%
K-Nearest Neighbours	81.30%	79.25%	98.82%	87.96%
Baseline — approve every application	*69.11%*	*69.11%*	*100%*	*81.73%*

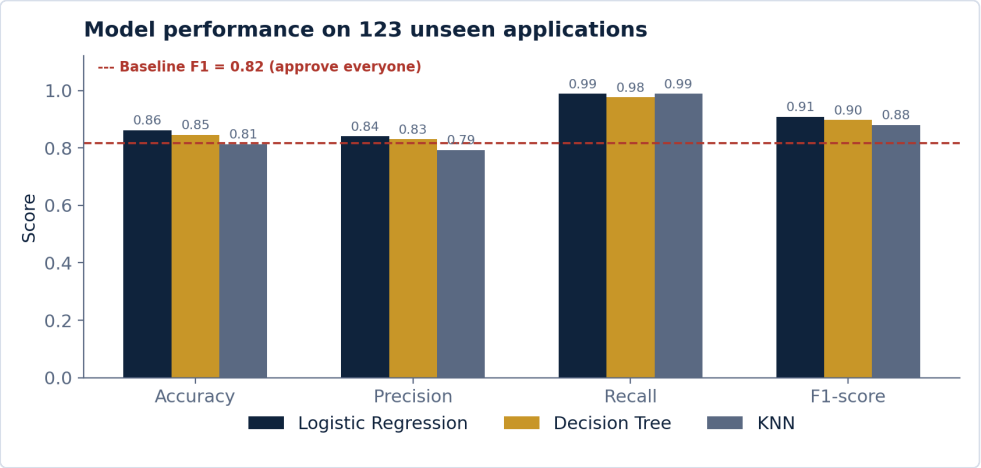
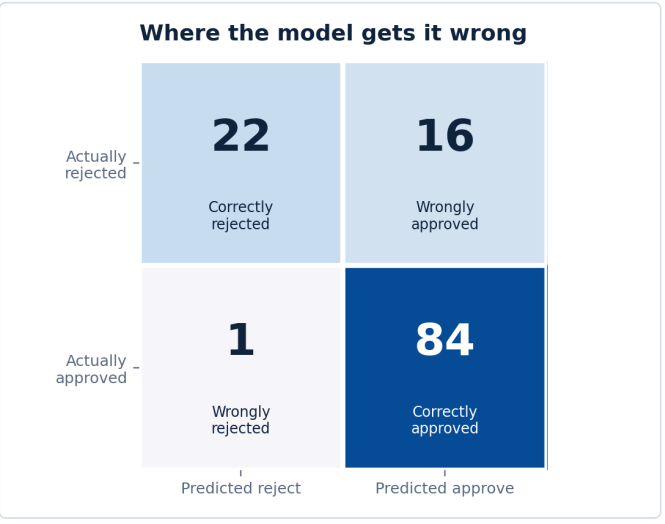


Figure 3. All three models beat the naive baseline, confirming genuine signal in the data.

Where the recommended model errs

Of 123 test applications, Logistic Regression classified 106 correctly. It found 84 of 85 applications that deserved approval, but caught only 22 of 38 that should have been declined — 16 applications were wrongly approved, the commercially costly mistake.

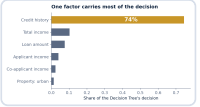


08

Key Findings

- **Logistic Regression is recommended** — not because every number is the highest, but because it is explainable, stable, and easy to audit, with the strongest precision on the error that costs a lender most.
- **Credit history explains almost the entire decision.** Figure 4 shows it accounts for roughly three-quarters of the Decision Tree's decision-making.
- Logistic Regression and the Decision Tree perform closely (within two applications out of 123) — a difference too small to call a strong result on its own.

Figure 4 (right) confirms this numerically: credit history alone accounts for roughly three-quarters of the Decision Tree's decision-making, with total income, loan amount and everything else a distant second.



09

Limitations

Limitation	Consequence
Single-factor dependence	The model is close to the rule “check the credit history”. Where that field is missing or wrong, it is unreliable.
Weak on rejections	Missed 16 of 38 applications that should have been declined — the costliest error type for a lender.
Small sample	123 test applications — differences of one or two applications between models are not meaningful.
Imputed data in the decisive field	50 records had no recorded credit history and were filled with the most common value — an assumption that carries real weight here.
Predicts past decisions, not repayment	The model learns what officers *did*, not who actually repaid. Any bias in past decisions is reproduced, not corrected.
Fairness exposure	Gender and marital status are inputs and 80% of applicants are male — these fields would need a fairness review before real deployment.

10

Conclusion

Loan approval **can** be predicted reliably enough to screen routine applications — all three models clearly beat the 69% baseline. But the more valuable finding is not the accuracy figure: it is that **credit history alone drives almost the entire decision**, with income and other demographics adding very little. Logistic Regression is the recommended model for its balance of performance, stability and explainability, with the caveat that no model here should replace human judgement on rejections, where the cost of error is highest.

11

Reflection

The part of this project I am proudest of is not the first version I submitted — it is the version that came after review. My first pipeline filled missing values across the whole dataset before splitting it into train and test, and I chose k for KNN by checking which value scored best directly on the test set. Both looked completely reasonable to me at the time, and the notebook ran without any error. It was only through review that I learned **a bug in a machine learning pipeline does not always throw an error — sometimes it just quietly hands you a better-looking number than you earned**. That was the single most important thing I learned in this project, more than any specific line of code.

Fixing it changed how I think about building a model, not just this one. I had to learn to ask, for every step: *could this step have seen information it shouldn't have?* Rebuilding the cleaning and modelling stages around a single `ColumnTransformer` and `Pipeline`, fitted only on the training fold and reused automatically inside cross-validation, was the fix — and applying that same discipline to **all three** models, not just the one that was flagged, felt like the difference between patching a mistake and actually understanding why it happened.

I was also surprised by how little income mattered once credit history was in the picture — I expected a stronger relationship and had to let the data correct my assumption rather than the other way round. If I did this project again, I would build the leakage-safe pipeline from the very first draft, and I would treat "my code runs and the score looks good" as the *start* of checking my work, not the end of it.

— Abdulkadir Abdullahi Mohamed

Source: Loan Prediction Dataset, kaggle.com/datasets/ssiddharth408/loan-prediction-dataset, public educational dataset. **Tools:** Python 3 — pandas, numpy, matplotlib, seaborn, scikit-learn. Full code and methodology are documented in the accompanying Jupyter notebook.